Software engineering

Software engineering is a branch of both computer science and engineering focused on designing, developing, testing, and maintaining software applications. It involves applying engineering principles and computer programming expertise to develop software systems that meet user needs.

A software engineer applies a software development process to define, implement, test, manage, and maintain software systems.

Beginning in the 1960s, software engineering was recognized as a separate field of engineering.

The development of software engineering was seen as a struggle. Problems included software that was over budget, exceeded deadlines, required extensive debugging and maintenance, and unsuccessfully met the needs of consumers or was never even completed.

In 1968, NATO organized the first conference on software engineering, which addressed emerging challenges in software development

and played a key role in formalizing guidelines and best practices for creating reliable and maintainable software.

The origins of the term software engineering have been attributed to various sources. The term appeared in a list of services offered by companies in the June 1965 issue of "Computers and Automation" and was used more formally in the August 1966 issue of Communications of the ACM (Volume 9, number 8) in "President's Letter to the ACM Membership" by Anthony A. Oettinger. It is also associated with the title of a NATO conference in 1968 by Professor Friedrich L. Bauer. Margaret Hamilton described the discipline of "software engineering" during the Apollo missions to give what they were doing legitimacy. At the time, there was perceived to be a "software crisis". The 40th International Conference on Software Engineering (ICSE 2018) celebrated 50 years of "Software Engineering" with the Plenary Sessions' keynotes of Frederick Brooks and Margaret Hamilton.

In 1984, the Software Engineering Institute (SEI) was established as a federally funded research and development center headquartered on the campus of Carnegie Mellon University in Pittsburgh, Pennsylvania, United States.

Watts Humphrey founded the SEI Software Process Program, aimed at understanding and managing the software engineering process. The Process Maturity Levels introduced became the Capability Maturity Model Integration for Development (CMMI-DEV), which defined how the US Government evaluates the abilities of a software development team.

Modern, generally accepted best practices for software engineering have been collected by the ISO/IEC JTC 1/SC 7 subcommittee and published as the Software Engineering Body of Knowledge (SWEBOK). Software engineering is considered one of the major computing disciplines.

In modern systems, where concepts such as Edge Computing, Internet of Things and Cyber-physical Systems are prevalent, software is a critical factor. Thus, software engineering is closely related to the systems engineering discipline. The Systems Engineering Body of Knowledge claims:

Software is prominent in most modern systems architectures and is often the primary means for integrating complex system components. Software engineering and systems engineering are not merely related disciplines; they are intimately intertwined.... Good systems engineering is a key factor in enabling good software engineering.

Notable definitions of software engineering include:

"The systematic application of scientific and technological knowledge, methods, and experience to the design, implementation, testing, and documentation of software."—The Bureau of Labor Statistics—IEEE Systems and software engineering – Vocabulary

"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software."—IEEE Standard Glossary of Software Engineering Terminology

"An engineering discipline concerned with all aspects of software production." — Ian Sommerville

"The establishment and use of sound engineering principles in order to economically obtain software that is reliable and works efficiently on real machines."—Fritz Bauer

"A branch of computer science that deals with the design, implementation, and maintenance of complex computer programs."—Merriam-Webster

"'Software engineering' encompasses not just the act of writing code, but all of the tools and processes an organization uses to build and maintain that code over time. [...] Software engineering can be thought of as 'programming integrated over time.'"—Software Engineering at Google

The term has also been used less formally:

As the informal contemporary term for the broad range of activities that were formerly called computer programming and systems analysis

As the broad term for all aspects of the practice of computer programming, as opposed to the theory of computer programming, which is formally studied as a sub-discipline of computer science

As the term embodying the advocacy of a specific approach to computer programming, one that urges that it be treated as an engineering discipline rather than an art or a craft, and advocates the codification of recommended practices

Individual commentators have disagreed sharply on how to define software engineering or its legitimacy as an engineering discipline. David Parnas has said that software engineering is, in fact, a form of engineering. Steve McConnell has said that it is not, but that it should be. Donald Knuth has said that programming is an art and a science. Edsger W. Dijkstra claimed that the terms software engineering and software engineer have been misused in the United States.

=== Requirements analysis ===

Requirements engineering is about elicitation, analysis, specification, and validation of requirements for software. Software requirements can be functional, non-functional or domain.

Functional requirements describe expected behaviors (i.e. outputs). Non-functional requirements specify issues like portability, security, maintainability, reliability, scalability, performance, reusability, and flexibility. They are classified into the following types: interface constraints, performance constraints (such as response time, security, storage space, etc.), operating constraints, life cycle constraints (maintainability, portability, etc.), and economic constraints. Knowledge of how the system or software works is needed when it comes to specifying non-functional requirements. Domain requirements have to do with the characteristic of a certain category or domain of projects.

Software design is the process of making high-level plans for the software. Design is sometimes divided into levels:

Interface design plans the interaction between a system and its environment as well as the inner workings of the system.

Architectural design plans the major components of a system, including their responsibilities, properties, and interfaces between them.

Detailed design plans internal elements, including their properties, relationships, algorithms and data structures.

Software construction typically involves programming (a.k.a. coding), unit testing, integration testing, and debugging so as to implement the design."Software testing is related to, but different from, ... debugging".

Software testing is an empirical, technical investigation conducted to provide stakeholders with information about the quality of the software under test. Software testing can be viewed as a risk based activity.

When described separately from construction, testing typically is performed by test engineers or quality assurance instead of the programmers who wrote it. It is performed at the system level and is considered an aspect of software quality. The testers' goals during the testing process are to minimize the overall number of tests to a manageable set and make well-informed decisions regarding which risks should be prioritized for testing and which can wait.

=== Program analysis ===

Program analysis is the process of analyzing computer programs with respect to an aspect such as performance, robustness, and security.

Software maintenance refers to supporting the software after release. It may include but is not limited to: error correction, optimization, deletion of unused and discarded features, and enhancement of existing features.

Usually, maintenance takes up 40% to 80% of project cost.

Knowledge of computer programming is a prerequisite for becoming a software engineer. In 2004, the IEEE Computer Society produced the SWEBOK, which has been published as ISO/IEC Technical Report 1979:2005, describing the body of knowledge that they recommend to be mastered by a graduate software engineer with four years of experience.

Many software engineers enter the profession by obtaining a university degree or training at a vocational school. One standard international curriculum for undergraduate software engineering degrees was defined by the Joint Task Force on Computing Curricula of the IEEE Computer Society and the Association for Computing Machinery, and updated in 2014. A number of universities have Software Engineering degree programs; as of 2010, there were 244 Campus Bachelor of Software Engineering programs, 70 Online programs, 230 Masters-level programs, 41 Doctorate-level programs, and 69 Certificate-level programs in the United States.

In addition to university education, many companies sponsor internships for students wishing to pursue careers in information technology. These internships can introduce the student to real-world tasks that typical software engineers encounter every day. Similar experience can be gained through military service in software engineering.

=== Software engineering degree programs ===

A small but growing number of practitioners have software engineering degrees. In 1987, the Department of Computing at Imperial College London introduced the first three-year software engineering bachelor's degree in the world; in the following year, the University of Sheffield established a similar program. In 1996, the Rochester Institute of Technology established the first software engineering bachelor's degree program in the United States; however, it did not obtain ABET accreditation until 2003, the same year as Rice University, Clarkson University, Milwaukee School of Engineering, and Mississippi State University.

Since then, software engineering undergraduate degrees have been established at many universities. A standard international curriculum for undergraduate software engineering degrees, SE2004, was defined by a steering committee between 2001 and 2004 with funding from the Association for Computing Machinery and the IEEE Computer Society. As of 2004, about 50 universities in the U.S. offer software engineering degrees, which teach both computer science and engineering principles and practices. The first software engineering master's degree was established at Seattle University in 1979. Since then, graduate software engineering degrees have been made available from many more universities. Likewise in Canada, the Canadian Engineering Accreditation Board (CEAB) of the Canadian Council of Professional Engineers has recognized several software engineering programs.

Additionally, many online advanced degrees in Software Engineering have appeared such as the Master of Science in Software Engineering (MSE) degree offered through the Computer Science and Engineering Department at California State University, Fullerton. Steve McConnell opines that because most universities teach computer science rather than software engineering, there is a shortage of true software engineers. ETS (École de technologie supérieure) University and UQAM (Université du Québec à Montréal) were mandated by IEEE to develop the Software Engineering Body of Knowledge (SWEBOK), which has become an ISO standard describing the body of knowledge covered by a software engineer.

Legal requirements for the licensing or certification of professional software engineers vary around the world. In the UK, there is no licensing or legal requirement to assume or use the job title Software Engineer. In some areas of Canada, such as Alberta, British Columbia, Ontario, and Quebec, software engineers can hold the Professional Engineer (P.Eng) designation and/or the Information Systems Professional (I.S.P.) designation. In Europe, Software Engineers can obtain the European Engineer (EUR ING) professional title. Software Engineers can also become professionally qualified as a Chartered Engineer through the British Computer Society.

In the United States, the NCEES began offering a Professional Engineer exam for Software Engineering in 2013, thereby allowing Software Engineers to be licensed and recognized. NCEES ended the exam after April 2019 due to lack of participation. Mandatory licensing is currently still largely debated, and perceived as controversial.

The IEEE Computer Society and the ACM, the two main US-based professional organizations of software engineering, publish guides to the profession of software engineering. The IEEE's Guide to the Software Engineering Body of Knowledge – 2004 Version, or SWEBOK, defines the field and describes the knowledge the IEEE expects a practicing software engineer to have. The most current version is SWEBOK v4. The IEEE also promulgates a "Software Engineering Code of Ethics".

There are an estimated 26.9 million professional software engineers in the world as of 2022, up from 21 million in 2016.

Many software engineers work as employees or contractors. Software engineers work with businesses, government agencies (civilian or military), and non-profit organizations. Some software engineers work for themselves as freelancers. Some organizations have specialists to perform each of the tasks in the software development process. Other organizations require software engineers to do many or all of them. In large projects, people may specialize in only one role. In small projects, people may fill several or all roles at the same time. Many companies hire interns, often university or college students during a summer break, or externships. Specializations include analysts, architects, developers, testers, technical support, middleware analysts, project managers, software product managers, educators, and researchers.

Most software engineers and programmers work 40 hours a week, but about 15 percent of software engineers and 11 percent of programmers worked more than 50 hours a week in 2008. Potential injuries in these occupations are possible because like other workers who spend long periods sitting in front of a computer terminal typing at a keyboard, engineers and programmers are susceptible to eyestrain, back discomfort, Thrombosis, Obesity, and hand and wrist problems such as carpal tunnel syndrome.

==== United States ====

The U. S. Bureau of Labor Statistics (BLS) counted 1,365,500 software developers holding jobs in the U.S. in 2018. Due to its relative newness as a field of study, formal education in software engineering is often taught as part of a computer science curriculum, and many software engineers hold computer science degrees. The BLS estimates 2024 to 2034 the growth for software engineers is 15% which is less than their prediction from 2023 to 2033 that computer software engineering would increase by 17%. This is down from the 2022 to 2032 BLS estimate of 25% for software engineering. And, is further down from their 30% 2010 to 2020 BLS estimate. Due to this trend, job growth may not be as fast as during the last decade, as jobs that would have gone to

computer software engineers in the United States would instead be outsourced to computer software engineers in countries such as India and other foreign countries. In addition, the BLS Job Outlook for Computer Programmers, the U.S. Bureau of Labor Statistics (BLS) Occupational Outlook predicts a decline of -7 percent from 2016 to 2026, a further decline of -9 percent from 2019 to 2029, a decline of -10 percent from 2021 to 2031. and then a decline of -11 percent from 2022 to 2032. Currently their prediction for 2024 to 2034 is a decline of -6 percent. Since computer programming can be done from anywhere in the world, companies sometimes hire programmers in countries where wages are lower. Furthermore, the ratio of women in many software fields has also been declining over the years as compared to other engineering fields. Then there is the additional concern that recent advances in Artificial Intelligence might impact the demand for future generations of Software Engineers. However, this trend may change or slow in the future as many current software engineers in the U.S. market flee the profession or age out of the market in the next few decades.

=== Certification ===

The Software Engineering Institute offers certifications on specific topics like security, process improvement and software architecture. IBM, Microsoft and other companies also sponsor their own certification examinations. Many IT certification programs are oriented toward specific technologies, and managed by the vendors of these technologies. These certification programs are tailored to the institutions that would employ people who use these technologies.

Broader certification of general software engineering skills is available through various professional societies. As of 2006, the IEEE had certified over 575 software professionals as a Certified Software Development Professional (CSDP). In 2008 they added an entry-level certification known as the Certified Software Development Associate (CSDA). The ACM and the IEEE Computer Society together examined the possibility of licensing of software engineers as Professional Engineers in the 1990s,

but eventually decided that such licensing was inappropriate for the professional industrial practice of software engineering. John C. Knight and Nancy G. Leveson presented a more balanced analysis of the licensing issue in 2002.

In the U.K. the British Computer Society has developed a legally recognized professional certification called Chartered IT Professional (CITP), available to fully qualified members (MBCS). Software engineers may be eligible for membership of the British Computer Society or Institution of Engineering and Technology and so qualify to be considered for Chartered Engineer status through either of those institutions. In Canada the Canadian Information Processing Society has developed a legally recognized professional certification called Information Systems Professional (ISP). In Ontario, Canada, Software Engineers who graduate from a Canadian Engineering Accreditation Board (CEAB) accredited program, successfully complete PEO's (Professional Engineers Ontario) Professional Practice Examination (PPE) and have at least 48 months of acceptable engineering experience are eligible to be licensed through the Professional Engineers Ontario and can become Professional Engineers P.Eng. The PEO does not recognize any online or distance education however; and does not consider Computer Science programs to be equivalent to software engineering programs despite the tremendous overlap between the two. This has sparked controversy and a certification war. It has also held the number of P.Eng holders for the profession exceptionally low. The vast majority of working professionals in the field hold a degree in CS, not SE. Given the difficult certification path for holders of non-SE degrees, most never bother to pursue the license.

=== Impact of globalization ===

The initial impact of outsourcing, and the relatively lower cost of international human resources in developing third world countries led to a massive migration of software development activities from corporations in North America and Europe to India and later: China, Russia, and other developing countries. This approach had some flaws, mainly the distance / time zone difference that prevented human interaction between clients and developers and the massive job transfer. This had a negative impact on many aspects of the software engineering profession. For example, some

students in the developed world avoid education related to software engineering because of the fear of offshore outsourcing (importing software products or services from other countries) and of being displaced by foreign visa workers. Additionally, the glut of high-tech workers has led to a wider adoption of the 996 working hour system and ‘007’ schedules as the expected work load. Although statistics do not currently show a threat to software engineering itself; a related career, computer programming does appear to have been affected. Nevertheless, the ability to smartly leverage offshore and near-shore resources via the follow-the-sun workflow has improved the overall operational capability of many organizations. When North Americans leave work, Asians are just arriving to work. When Asians are leaving work, Europeans arrive to work. This provides a continuous ability to have human oversight on business-critical processes 24 hours per day, without paying overtime compensation or disrupting a key human resource, sleep patterns.

While global outsourcing has several advantages, global – and generally distributed – development can run into serious difficulties resulting from the distance between developers. This is due to the key elements of this type of distance that have been identified as geographical, temporal, cultural and communication (that includes the use of different languages and dialects of English in different locations). Research has been carried out in the area of global software development over the last 15 years and an extensive body of relevant work published that highlights the benefits and problems associated with the complex activity. As with other aspects of software engineering research is ongoing in this and related areas.

There are various prizes in the field of software engineering:

ACM-AAAI Allen Newell Award- USA. Awarded to career contributions that have breadth within computer science, or that bridge computer science and other disciplines.

BCS Lovelace Medal. Awarded to individuals who have made outstanding contributions to the understanding or advancement of computing.

ACM SIGSOFT Outstanding Research Award, selected for individual(s) who have made "significant and lasting research contributions to the theory or practice of software engineering."

More ACM SIGSOFT Awards.

The Codie award, a yearly award issued by the Software and Information Industry Association for excellence in software development within the software industry.

Harlan Mills Award for "contributions to the theory and practice of the information sciences, focused on software engineering".

ICSE Most Influential Paper Award.

Jolt Award, also for the software industry.

Stevens Award given in memory of Wayne Stevens.

Some call for licensing, certification and codified bodies of knowledge as mechanisms for spreading the engineering knowledge and maturing the field.

Some claim that the concept of software engineering is so new that it is rarely understood, and it is widely misinterpreted, including in software engineering textbooks, papers, and among the communities of programmers and crafters.

Some claim that a core issue with software engineering is that its approaches are not empirical enough because a real-world validation of approaches is usually absent, or very limited and hence software engineering is often misinterpreted as feasible only in a "theoretical environment."

Edsger Dijkstra, a founder of many of the concepts in software development today, rejected the idea of "software engineering" up until his death in 2002, arguing that those terms were poor analogies for what he called the "radical novelty" of computer science:

A number of these phenomena have been bundled under the name "Software Engineering". As economics is known as "The Miserable Science", software engineering should be known as "The Doomed Discipline", doomed because it cannot even approach its goal since its goal is self-contradictory. Software engineering, of course, presents itself as another worthy cause, but that is eyewash: if you carefully read its literature and analyse what its devotees actually do, you will discover that software engineering has accepted as its charter "How to program if you cannot."

=== Study and practice ===

Algorithm engineering

Software craftsmanship

=== Professional aspects ===

Bachelor of Science in Information Technology

Bachelor of Software Engineering

List of software engineering conferences

List of computer science journals (including software engineering journals)

List of software programming journals

Lists of programming software development tools

Software Engineering Institute

== Further reading ==

Pierre Bourque; Richard E. (Dick) Fairley, eds. (2014). Guide to the Software Engineering Body of Knowledge Version 3.0 (SWEBOK). IEEE Computer Society.

Roger S. Pressman; Bruce Maxim (January 23, 2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill. ISBN 978-0-07-802212-8.

Ian Sommerville (March 24, 2015). Software Engineering (10th ed.). Pearson Education Limited. ISBN 978-0-13-394303-0.

Jalote, Pankaj (2005) [1991]. An Integrated Approach to Software Engineering (3rd ed.). Springer. ISBN 978-0-387-20881-7.

Bruegge, Bernd; Dutoit, Allen (2009). Object-oriented software engineering : using UML, patterns, and Java (3rd ed.). Prentice Hall. ISBN 978-0-13-606125-0.

Oshana, Robert (2019-06-21). Software engineering for embedded systems : methods, practical techniques, and applications (Second ed.). Kidlington, Oxford, United Kingdom. ISBN 978-0-12-809433-4.

Pierre Bourque; Richard E. Fairley, eds. (2004). Guide to the Software Engineering Body of Knowledge Version 3.0 (SWEBOK), https://www.computer.org/web/swebok/v3. IEEE Computer Society.

The Open Systems Engineering and Software Development Life Cycle Framework Archived 2010-07-18 at the Wayback Machine OpenSDLC.org the integrated Creative Commons SDLC

Software Engineering Institute Carnegie Mellon